

EX 10 IMAGE GENERATION USING A GENERATIVE ADVERSARIAL NETWORK (GAN)

AIM

To use a Generative Adversarial Network (StyleGAN2-ADA) to generate new images and evaluate their quality by visual inspection and a quantitative metric, the Fréchet Inception Distance (FID).

PROCEDURE

1. Clone the StyleGAN2-ADA repository and install the required packages.
2. Download the pre-trained FFHQ StyleGAN2 model and load the generator (G_ema).
3. Generate 20 images from random latent vectors using truncation $\psi = 0.7$ and save them.
4. Display the generated images in a grid for visual inspection.
5. Prepare 20 real images (CIFAR-10 resized to 64×64) as the reference set.
6. Compute the FID between the real and generated image folders using pytorch-fid.

ALGORITHM

1. Start.
2. Load the pre-trained StyleGAN2 generator.
3. Repeat 20 times: sample $z \sim N(0, I)$, pass it with an empty label through the generator (truncation 0.7), rescale the output to 0–1 and save it.
4. Display the generated images.
5. Save 20 real reference images.
6. Pass both sets through InceptionV3, compute feature mean and covariance, and calculate $FID = \|\mu_1 - \mu_2\|^2 + \text{Tr}(\Sigma_1 + \Sigma_2 - 2(\Sigma_1 \Sigma_2)^{1/2})$.
7. Print the FID score.
8. Stop.

PROGRAM

```
# =====
# EX 10 - IMAGE GENERATION USING GAN
# Fast Colab Version
# =====

# 1. Install StyleGAN2-ADA
!git clone -q https://github.com/NVlabs/stylegan2-ada-pytorch.git
!pip -q install ninja torchmetrics[image] pytorch-fid

# 2. Imports
import os
import sys
import urllib.request
import torch
import matplotlib.pyplot as plt

from torchvision.utils import save_image
from torchvision.io import read_image
```

```

sys.path.append("/content/stylegan2-ada-pytorch")

import legacy

# GPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)

# =====

# 3. Download pretrained StyleGAN2 model
# =====

url = "https://nvlabs-fi-cdn.nvidia.com/stylegan2-ada-pytorch/pretrained/ffhq.pkl"
model_path = "/content/ffhq.pkl"

if not os.path.exists(model_path):
    print("Downloading model...")
    urllib.request.urlretrieve(url, model_path)

print("Model ready!")

# =====
# 4. Load Generator
# =====

G = legacy.load_network_pkl(
    open(model_path, "rb")
) ["G_ema"].to(device)

# =====
# 5. Generate Images
# =====

NUM_IMAGES = 20

os.makedirs("/content/fake", exist_ok=True)

for i in range(NUM_IMAGES):

    z = torch.randn(
        [1, G.z_dim],
        device=device
    )

    label = torch.zeros(
        [1, G.c_dim],
        device=device
    )

    with torch.no_grad():
        img = G(
            z,
            label,
            truncation_psi=0.7,
            noise_mode="const"
        )

    img = (img + 1) / 2
    img = img.clamp(0, 1)

```

```

        save_image(
            img,
            f"/content/fake/fake_{i}.png"
        )

print("Generated", NUM_IMAGES, "images!")

# =====
# 6. Display Generated Images
# =====

fig, axes = plt.subplots(2, 5, figsize=(15, 6))

for i, ax in enumerate(axes.flat):

    img = read_image(
        f"/content/fake/fake_{i}.png"
    ).permute(1, 2, 0)

    ax.imshow(img)
    ax.axis("off")

plt.show()

# =====
# 7. Create Real Images
# =====
# Instead of downloading huge CelebA,
# use CIFAR-10 for a quick demonstration.

import torchvision
from torchvision import datasets, transforms

transform = transforms.Compose([
    transforms.Resize((64, 64)),
    transforms.ToTensor()
])

real_dataset = datasets.CIFAR10(
    root="/content/data",
    train=True,
    download=True,
    transform=transform
)

os.makedirs("/content/real", exist_ok=True)

for i in range(NUM_IMAGES):

    img, label = real_dataset[i]

    save_image(
        img,
        f"/content/real/real_{i}.png"
    )

print("Real images prepared!")

# =====
# 8. Calculate FID
# =====

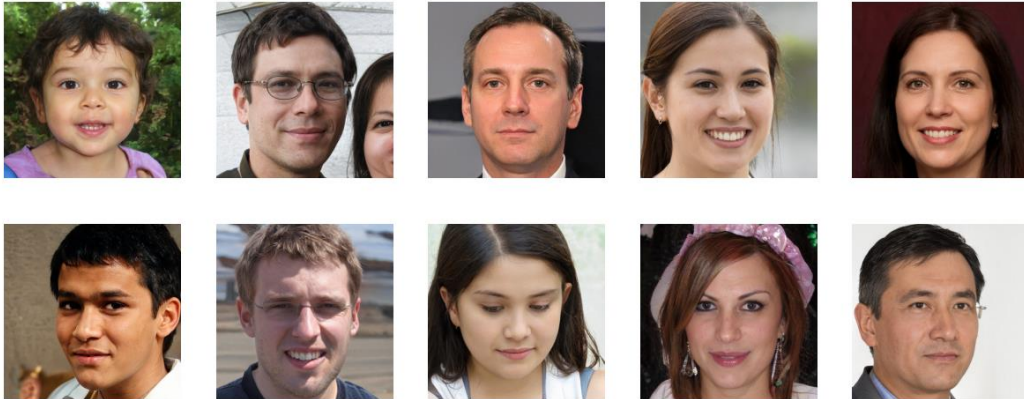
```

```
!python -m pytorch_fid /content/real /content/fake
```

OUTPUT

```
Device: cuda
Downloading model...
Model ready!
Generated 20 images!
Real images prepared!
FID: 453.20509584431977
```

Console output



Images generated by StyleGAN2

RESULT

Twenty face images were generated. The measured FID was 453.21; this is high because only 20 images were compared and the real set was CIFAR-10 (not faces), so the score should be read as a demonstration of the metric rather than a quality verdict.